



SYSTEME D'EXPLOITATION

PROJET:

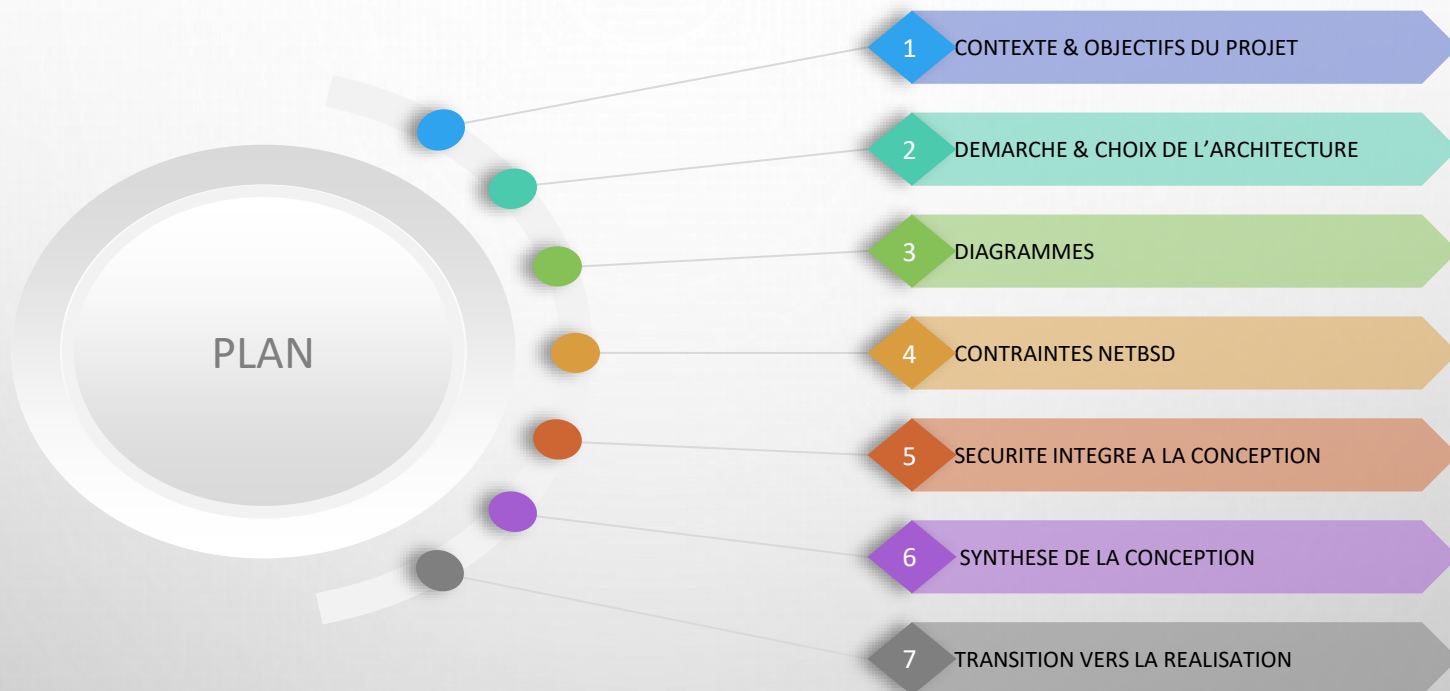
**Conception et
développement de RAM
Dumper, outil
d'extraction contrôlée du
contenu de la mémoire
vive sous NetBSD**

EQUIPE :

- SIMO SOH Yannick Dylan
- TEMGOUA NJOUNANG
Cedric Lionnel

**SOUS LA SUPERVISION DE
M. NGUIMBUS**

CAHIER DE CONCEPTION





CONTEXTE & OBJECTIF DU PROJET

Contexte

Dans le cadre de la **sécurité informatique** et de l'**investigation numérique**, l'analyse de la mémoire vive constitue une source essentielle d'informations (processus actifs, rootkits, clés de chiffrement).

Cependant, sous NetBSD, peu d'outils proposent:

- ✓ Une extraction mémoire sécurisée
- ✓ Une interface web
- ✓ Une gestion multi-utilisateurs

👉 Le projet RAM DUMPER répond à ce besoin.

Objectifs du cahier de conception

- ✓ Définir l'architecture logicielle de RAM DUMPER
- ✓ Décrire l'organisation interne du système
- ✓ Modéliser les interactions principales
- ✓ Intégrer les contraintes NetBSD et sécurité dès la conception

👉 Ce document prépare la phase de réalisation.



DEMARCHE DE CONCEPTION ADOPTÉE ET CHOIX DE L'ARCHITECTURE GLOBALE

Méthodologie

- ✓ Approche orientée architecture
- ✓ Modélisation UML
- ✓ Séparation claire des responsabilités
- ✓ Sécurité intégrée dès la conception (security by design)

Diagrammes utilisés



Diagramme de
classes



Diagramme de
séquence

Architecture retenue

- ✓ Architecture Client – Serveur
- ✓ Application Web sécurisée
- ✓ Backend natif C/C++ sous NetBSD
- ✓ Accès contrôlé à la mémoire via /dev/mem

Justification

- ✓ Accès distant sécurisé
- ✓ Centralisation des contrôles
- ✓ Meilleure traçabilité
- ✓ Gestion multi-utilisateurs



DIAGRAMME - DE CLASSES



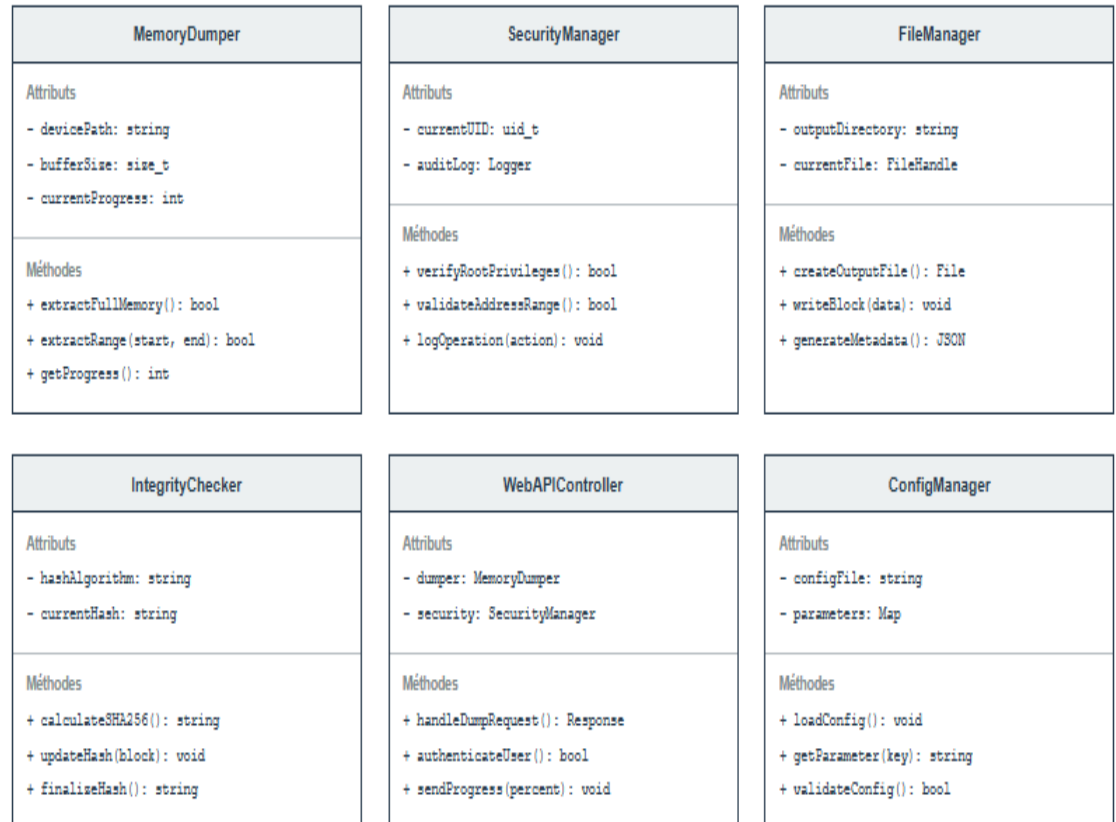
Objectif du diagramme de classes

- ✓ Structurer le code de manière claire et maintenable
- ✓ Définir les responsabilités de chaque classe
- ✓ Faciliter la maintenance et l'évolution du système



Il décrit la structure interne du backend.

Diagramme de Classes





DESCRIPTION DES CLASSES MÉTIERS & RELATIONS ENTRE CLASSES

MemoryDumper

- Orchestration de l'extraction mémoire
- Coordination des modules
- Gestion du workflow complet

SecurityManager

- Vérification UID = 0 (privileges root)
- Validation des plages mémoire
- Gestion des permissions et autorisations
- Journalisation des accès

FileManager

- Création et écriture des fichiers dump
- Gestion du stockage sur disque
- Vérification de l'espace disponible

IntegrityChecker

- Calcul SHA-256 incrémental pendant l'extraction
- Garantie de l'intégrité des données

WebApiController

- Exposition des services REST
- Gestion de l'authentification JWT
- Validation des requêtes

ConfigManager

- Chargement de la configuration système
- Gestion des paramètres applicatifs

Relations principales

- 1 **WebApiController** → **MemoryDumper** : Orchestration
- 2 **MemoryDumper** → **SecurityManager** : Validation sécurité
- 3 **MemoryDumper** → **FileManager** : Gestion fichiers
- 4 **MemoryDumper** → **IntegrityChecker** : Calcul hash

Scénarios

MemoryDumper est la classe principale orchestrant l'extraction mémoire.

SecurityManager vérifie les privilèges root (UID=0) et valide les plages d'adresses avant toute opération.

FileManager gère la création et l'écriture progressive du fichier dump.

IntegrityChecker calcule le hash SHA-256 de manière incrémentale pendant l'écriture.

WebApiController expose les fonctionnalités via API REST et gère l'authentification JWT.

ConfigManager charge les paramètres de configuration depuis le fichier de configuration.



DIAGRAMME - DE SÉQUENCE

Objectif

- ✓ Décrire le comportement dynamique du système
- ✓ Visualiser l'ordre des traitements
- ✓ Illustrer un scénario clé d'utilisation

 Scénario choisi : Extraction mémoire complète

Acteurs impliqués



Client Web

Utilisateur interface



WebAPIController

Point d'entrée API



SecurityManager

Validation sécurité



MemoryDumper

Moteur extraction



FileManager

Gestion fichiers



IntegrityChecker

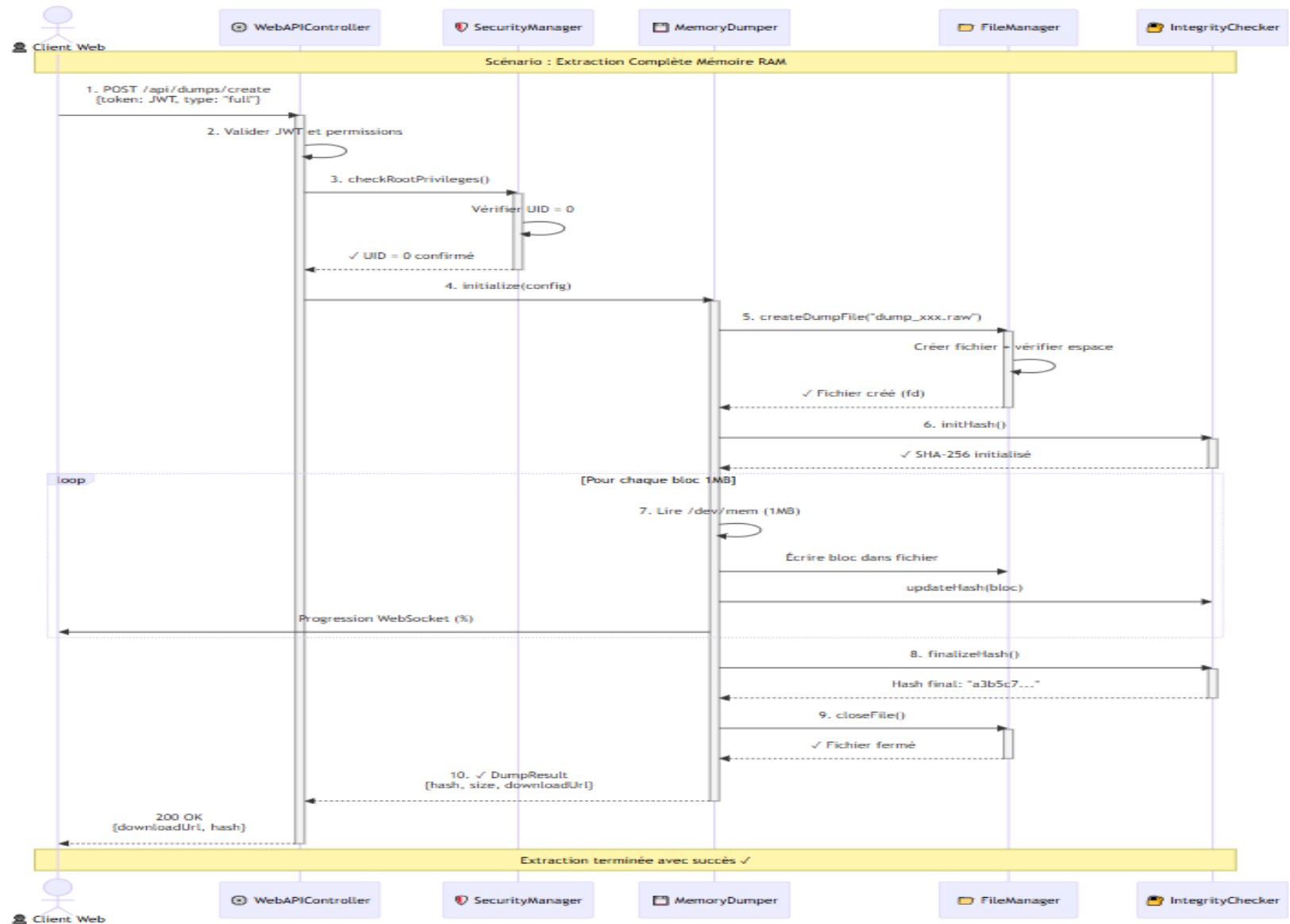
Hash SHA-256

Scénario d'extraction mémoire

- 1 Authentification utilisateur (JWT)
- 2 Vérification des permissions
- 3 Contrôle UID = 0 (privilèges root)
- 4 Ouverture /dev/mem en lecture seule
- 5 Lecture mémoire par blocs de 1MB
- 6 Écriture progressive dans fichier dump
- 7 Calcul SHA-256 incrémental
- 8 Envoi progression temps réel (WebSocket)
- 9 Génération lien de téléchargement



DIAGRAMME - DE SÉQUENCE





CONTRAINTES NETBSD



Contraintes techniques

- Accès /dev/mem en lecture seule uniquement
- Privilèges root obligatoires (UID = 0)
- Impact sur les performances à maîtriser
- Gestion des volumes de données importants



Solutions apportées

- Lecture par blocs de 1MB optimisée
- Validation stricte des permissions
- Bufferisation intelligente
- Compression optionnelle des dumps

SÉCURITÉ INTÉGRÉE À LA CONCEPTION

Authentification JWT

- ✓ Tokens avec expiration (1h)
- ✓ Refresh tokens sécurisés
- ✓ Signature cryptographique

RBAC (Contrôle d'accès)

- ✓ Rôles: Admin, Operator, Analyst, Viewer
- ✓ Permissions granulaires
- ✓ Principe du moindre privilège

Validation stricte

- ✓ Validation des entrées utilisateur
- ✓ Sanitization des données
- ✓ Protection injection SQL

Journalisation complète

- ✓ Logs syslog détaillés
- ✓ Audit trail complet
- ✓ Conservation 1 an minimum

 **Isolation des responsabilités pour limiter la surface d'attaque**



SYNTHÈSE DE LA CONCEPTION

Synthèse



Architecture claire et sécurisée

Séparation des responsabilités



Conception modulaire

Composants indépendants et réutilisables



Diagrammes UML cohérents

Documentation complète du système



Adaptée aux exigences forensic

Intégrité et traçabilité garanties



TRANSITION VERS LA RÉALISATION

Perspectives

- 1 **Implémentation des composants** selon l'architecture définie
- 2 **Développement backend C/C++** du module Memory Dumper
- 3 **Interface web** avec React et Material-UI
- 4 **Tests et validation** (unitaires, intégration, sécurité)

CONCLUSION

